Module 4

Goal based agents for Problem Solving : Planning
Planning : The planning problem, Planning Vs Searching, ADL, Partial order planning, Hierarachical task planning
#Self Learning – Multiagent planning, classical planning and real-world planning

What is planning in AI?

Planning in Artificial Intelligence is about the **decision making tasks performed by the robots** or computer programs to achieve a specific goal.

- The execution of planning is about **choosing a sequence of actions** with a high likelihood to complete the specific task.

Real-World Problems

- Planning: the task of coming up with a **sequence of action**s that will achieve a goal
  - Search-based problem-solving agent
  - Logical planning agent
  - Complex/large scale problems

- Consider only environments that are fully observable, deterministic, finite, static (change happens only when the agent acts), and discrete (in time, action, objects, and effects)
- **These are called classical planning environments.**
  - In contrast, non classical planning is for partially observable or stochastic environments and involves a different set of algorithms and agent designs

Searching v/s Planning

Feature	Searching	Planning
Focus	Finding path	Deciding actions
Intelligence level	Lower	Higher
Uses domain knowledge	No	Yes
Output	Path to goal	Action sequence
Example	Maze solving	Robot task execution

PLANNING

- TOP – Total Order Planning
- POP- Partial Order Planning

Total-Order Planning

- Only explore linear sequences of actions from start to goal state
- They <u>cannot take advantage of problem decomposition</u>, i.e. splitting the problem into smaller sub-problems and solving them individually.

Partial-Order Planning

- Works on problem decomposition.
- Divide the problem into parts and achieve these sub goals independently.
- S**olves the sub problems with sub plans** and then **combines these sub plans and reorders them based on requirements.**
- Ordering of the actions is partial.
- Does not specify which action will come first out of the two actions which are placed in the plan.

POP Example

- Putting on a pair of shoes:
- Goal(RightShoeOn ^ LeftShoeOn)
- Init()
- Action: RightShoe
  - PRECOND: RightSockOn
  - EFFECT: RightShoeOn
- Action: RightSock
  - PRECOND: None
  - EFFECT: RightSockOn
- Action:LeftShoe
  - PRECOND: LeftSockOn
  - EFFECT: LeftShoeOn
- Action: LeftSock
  - PRECOND: None
  - EFFECT: LeftSockOn

Init: Barefoot

Goal: RightShoeOn ^ LeftShoeOn

Action: 1. RightShoeOn

Precondition: RightSockOn

Effect: RightShoeOn

2. LeftShoeOn

Precondition: LeftSockOn

Effect: LeftShoeOn

3. LeftSockOn

Precondition: Barefoot

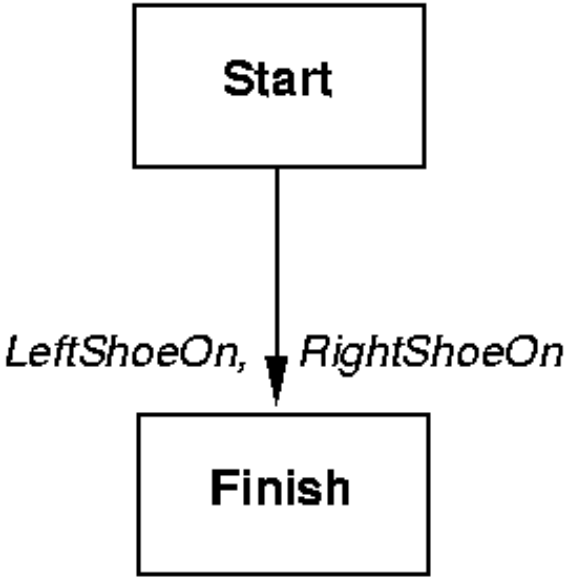
Effect: LeftSockOn

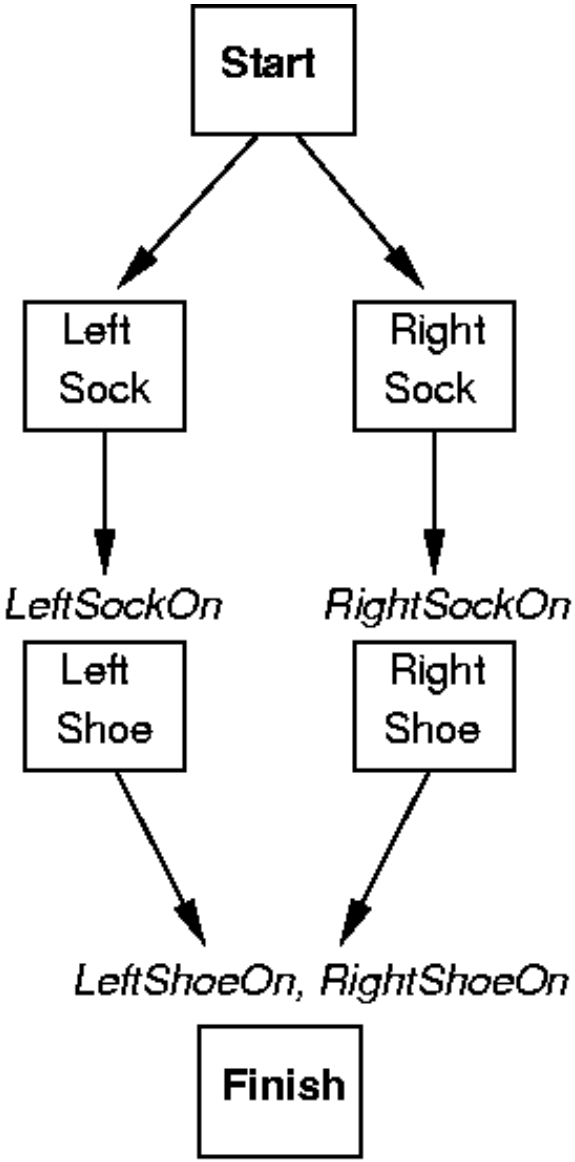
4. RightSockOn

Precondition: Barefoot

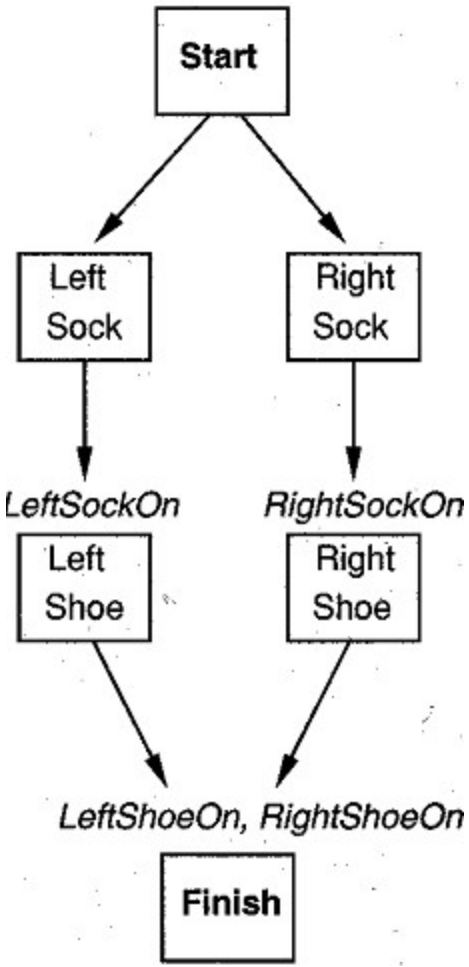
Effect: RightSockOn

POP Example

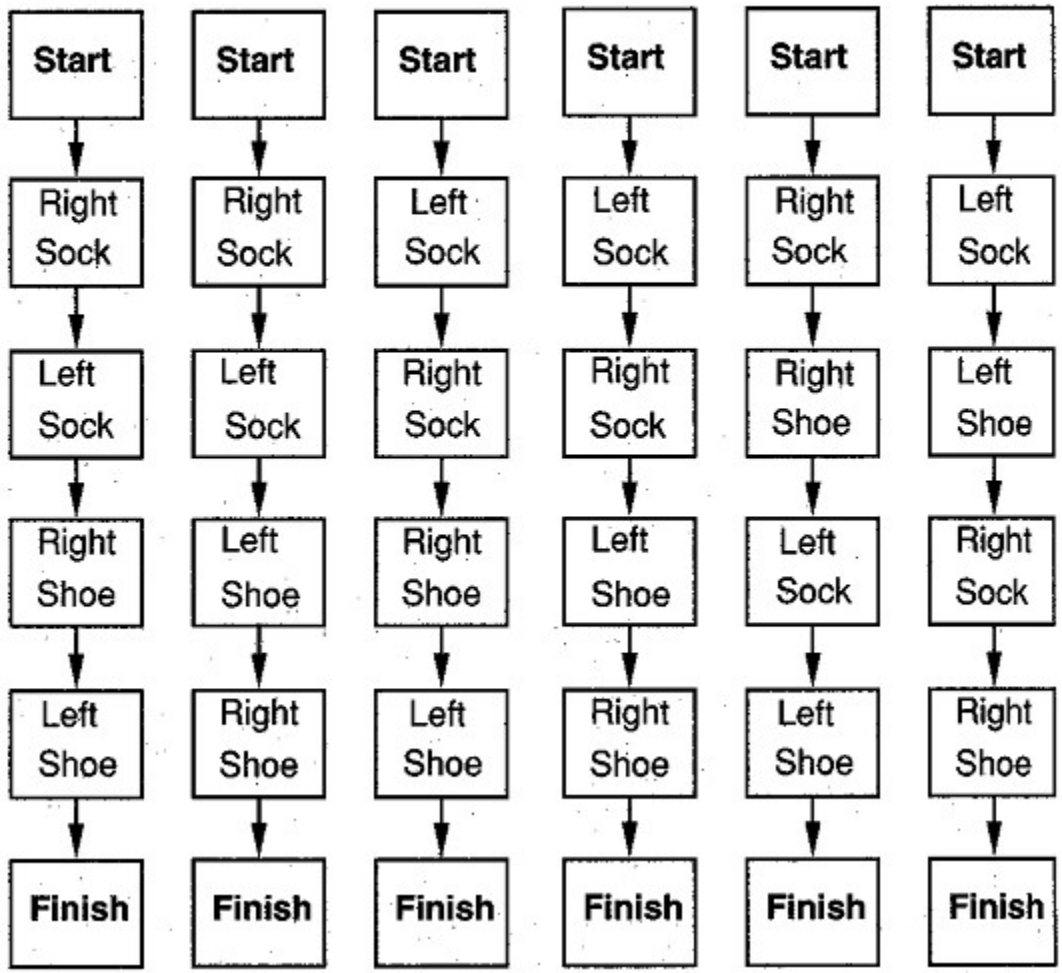

Start
LeftShoeOn, RightShoeOn
Finish


Start
Left Sock
Right Sock
LeftSockOn
RightSockOn
Left Shoe
Right Shoe
LeftShoeOn, RightShoeOn
Finish

Partial Order Plan:

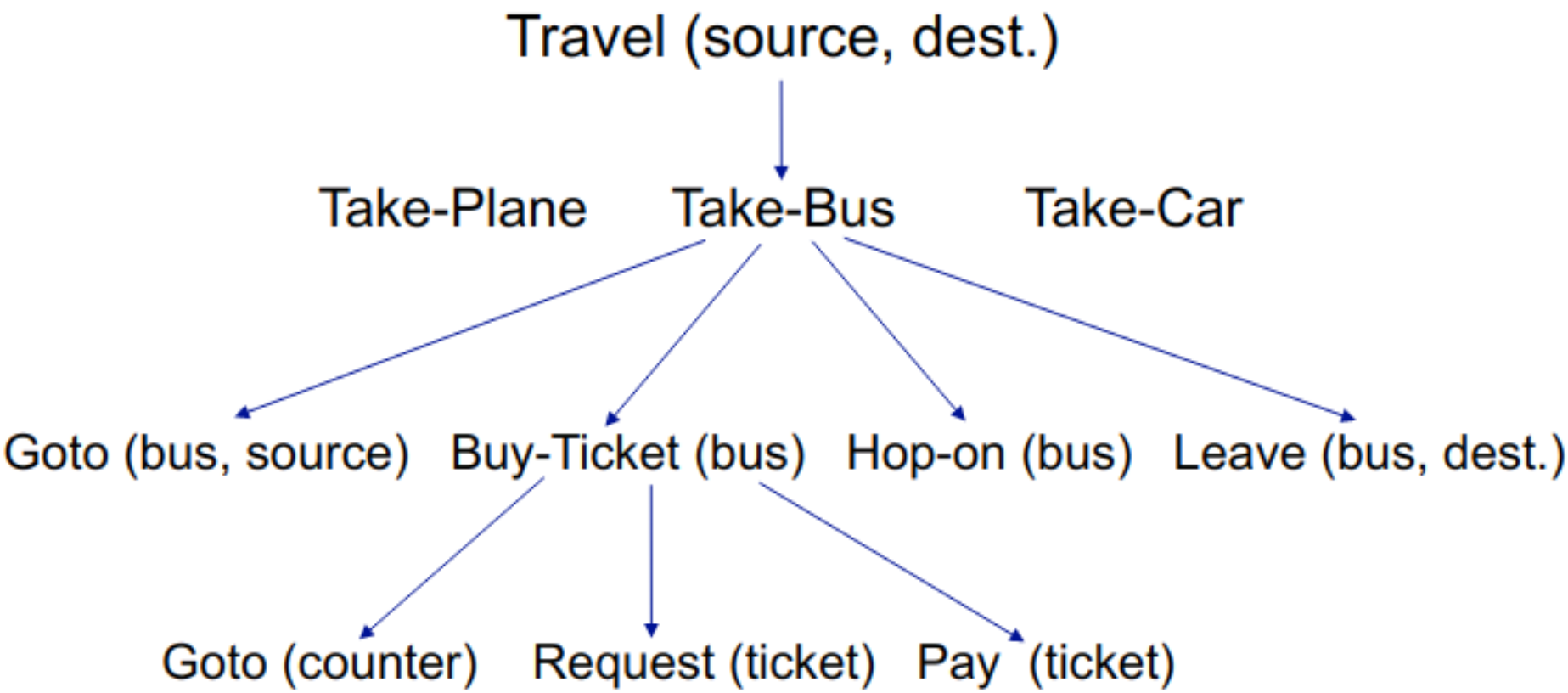


Total Order Plans:

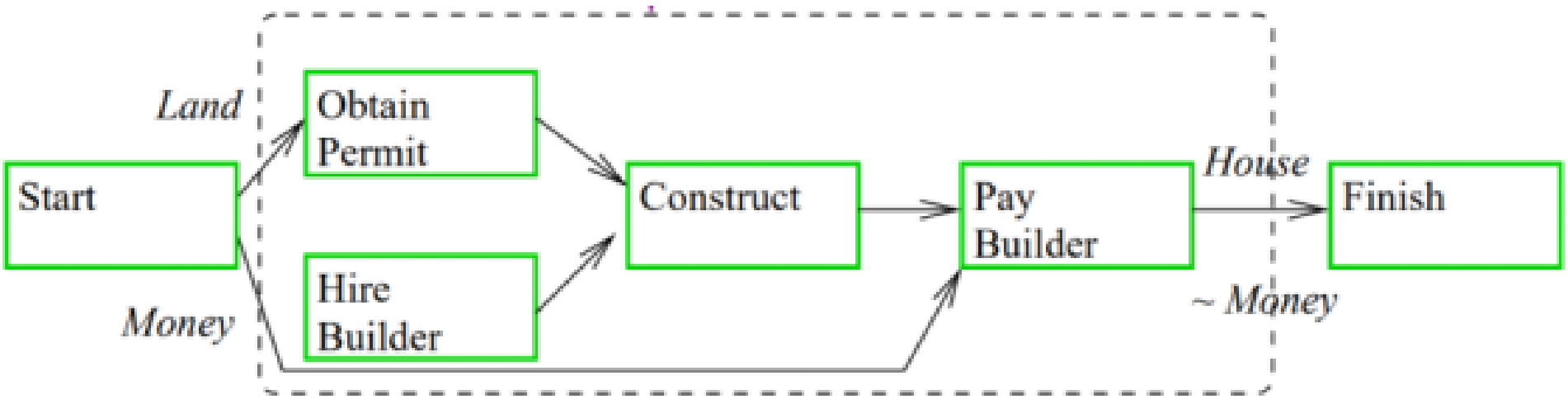


Hierarchical Planning

- Plans are organized in a hierarchical format.
- It works on plan decomposition.
- Complex actions are decomposed into simpler or primitive ones and it can be denoted with the help of links between various states at different levels of the hierarchy.


Travel (source, dest.)
Take-Plane
Take-Bus
Take-Car
Goto (bus, source)
Buy-Ticket (bus)
Hop-on (bus)
Leave (bus, dest.)
Goto (counter)
Request (ticket)
Pay (ticket)

For example
In case of building a house use of hierarchical planning



Action Description Language (ADL)

- AI Planning Language
- Used to describe actions and their effects
- Extension of STRIPS
- Used in Artificial Intelligence planning
- More expressive than STRIPS

Not in syllabus directly STRIPS-STanford Research Institute Problem Solver

- A set of **states-** It is a conjunction of positive ground literals.
- A set of **goals-** partially specified state represented as a conjunction of positive literals, and
- A set of **actions-** For each action, there is a precondition that must be satisfied and an effect which reflects the impact the action has on the environment after it has been performed.

Not in syllabus directly STRIPS

STRIPS (STanford Research Institute Problem Solver)

- a restrictive way to express states, actions and goals, but leads to more efficiency
- **States**: conjunctions of ground, function-free, and positive literals, such as At(Home) ^ Have(Banana)
- Closed-world assumption is used
- **Goals**: conjunctions of literals, may contain variables (existential), hence goal may represent more than one state
- E.g. At(Home) ^ ~ Have(Bananas)
- E.g. At(x) ^ Sells(x, Bananas)
- **Actions**: preconditions that must hold before execution and the effects after execution

Not in syllabus directly

STRIPS Action Schema

- An action schema includes:
- action name & parameter list (variables)

Example:

- Action: Buy (x)
- Precondition: At (p), Sells (p, x)
- Effect: Have(x)

Key Features of ADL

- Supports negative preconditions
- Allows disjunction (OR conditions)
- Conditional effects
- Quantifiers (forall, exists)

Components of ADL

- Actions
- Preconditions
- Effects
- State representation

Preconditions in ADL

- Conditions that must be true before an action
- Can include AND, OR, NOT
- More flexible than STRIPS

Effects in ADL

- Describes result of actions
- Can be conditional
- Adds or removes facts from state

Advantages of ADL

- More expressive than STRIPS
- Handles complex planning problems
- Supports real-world scenarios

Disadvantages of ADL

- More complex to implement
- Higher computational cost
- Harder to solve

Example of ADL

- Action: Move(A, B)
- Precondition: At(A) AND Path(A,B)
- Effect: At(B) AND NOT At(A)

Conclusion

- ADL is powerful planning language
- Used in AI for complex decision-making
- Extends basic planning models

SR. NO.	PARAMETERS	STRIPS	ADL
1	Stands for	**S**Tanford **R**esearch **I**nstitute **P**roblem **S**olver	**A**ction **D**escription **L**anguage
2	Literals allowed in states	Only Positive. E.g.: Intelligent /\ Beautiful	Positive as well as Negative. E.g.: ~Stupid /\ ~Ugly
3	Assumption for Unmentioned Literals	Closed World Assumptions: Unmentioned literals are False	Open World Assumptions: Unmentioned literals are Unknown
4	Equality	No Support	Equality predicate (x=y) is Built-in
5	Types	No Support	Supported. Variables can have types -> p:Person
6	Effects	Effects are conjunctions	Conditional Effects allowed (when P:E)
7	Effect P /\ ~Q means:	Add P & Delete Q	Add P /\ ~Q Delete ~P /\ Q
8	Goals	Goals are conjunctions	Allows conjunctions & disjunctions: ~Poor /\ (Famous \/ Smart)
9	Goals	Only ground literals	Quantified Variables
10	Schema includes	Action name Parameter list Precondition Effect	Action name Parameter list (optional) Groups of clauses labelled: Precond, Add, Delete and Update (optional)
11	Modelling actions in real world applications	Not Suitable	Suitable (This inadequacy of STRIPS led to development of ADL)